

SET IMPLEMENTATION QUESTIONS

1.) How set is implemented internally with which data structure?

→ implemented using a balanced binary search tree, typically a **Red-Black Tree**.

→ A **vector** or **array** doesn't automatically maintain **sorted order** when you insert elements. You'd have to sort it after every insertion, which would be costly.

A **priority queue** only gives you access to the largest or smallest element efficiently, but it doesn't let you search, erase, or check existence of arbitrary elements in $O(\log n)$ $O(\log n)$ $O(\log n)$.

A **balanced BST** like Red-Black Tree ensures:

- Insertion, deletion, search → $O(\log n)$ $O(\log n)$ $O(\log n)$
- Elements are always **sorted**
- Can iterate in **sorted order** efficiently

2.) Can you tell me the time complexity of iterating through all elements?

Also, what happens if you try to **modify an element** through an iterator?

→ **✗** But you **cannot modify an element through a set iterator**.

Reason: 1.) Elements in a **set** are **const** when accessed via iterator.

- This is because changing the value could **break the BST's ordering invariant**.
 - If you need to “modify” an element, you have to **erase it** and **insert the new value**.
-

`insert`, `erase`, `find` → all $O(\log n)$ because the underlying **BST** keeps elements sorted.

CODING IMPLEMENTATION

Step 1: How an interviewer would transition to implementation

Interviewer-style prompt:

“Assume you don’t have access to `std::set`.

I want you to build your own simplified `set` with the following operations:

- `insert(x)`
- `erase(x)`
- `exists(x)`

You can pick any internal data structure, but insertion and search should be efficient. How would you design it?”

How YOU should begin answering (interview-style)

Before jumping into code, you should **clarify the constraints and pick an approach**.

Your response should sound like this:  **What you should say:**

“First, I’ll clarify a few things:

- Do we require the elements to be stored in sorted order?

- Are duplicate insertions allowed?
- What's the expected time complexity for operations?
Depending on these constraints, I'll pick an underlying structure."

This shows maturity. Then, assuming the interviewer says:

- No duplicates
- Order matters (like `set`)
- Efficient insert/find/erase
You should say:

"I'll use a Binary Search Tree to implement the set. If balancing is required I'd use AVL/Red-Black Tree, but for now I'll implement an unbalanced BST for simplicity."

STEP -2 → Good. So if you're using a Binary Search Tree, how will you design the nodes and the overall class structure?"

Here's what you should respond with (and think like this in future interviews):

✅ Your expected answer:

"I'll create a Node struct with

`int value`

`Node* left`

`Node* right`

Then I'll create a MySet class which maintains:

`a Node* root`

And exposes the functions:

`insert(int x)`

`exists(int x)`

`erase(int x)"`

That's the right direction

CODE STRUCTURE:

```

struct Node {

    int val;

    Node* left;

    Node* right;

    Node(int v) : val(v), left(NULL), right(NULL) {}

};

class MySet {

private:

    Node* root;

public:

    MySet() : root(NULL) {}

    void insert(int x) {

        // to be implemented

    }

    bool exists(int x) {

        // to be implemented

    }

    void erase(int x) {

        // to be implemented    }

};

```

STEP 3

```

void insert(int x) {
    root = insertHelper(root, x);
}

Node* insertHelper(Node* node, int x) {
    if(node == NULL) {
        return new Node(x);
    }
    if(x < node->val) {
        node->left = insertHelper(node->left, x);
    }
    else if(x > node->val) {
        node->right = insertHelper(node->right, x);
    }
    // if x == node->val, do nothing (no duplicates)
    return node;
}

```

A.)

INSERT FUNCTION

o insert into the BST, I'll start at the root and compare the value:

- If the value is equal to the current node, I'll return because we don't allow duplicates.
- If it's smaller, I move to the left child.
- If it's larger, I move to the right child.

I keep going until I find a **NULL** position and create a new node there.

I'll implement this using a helper recursive function for cleaner code."

```

cpp

bool exists(int x) {
    return existsHelper(root, x);
}

bool existsHelper(Node* node, int x) {
    if(node == NULL) return false;
    if(node->val == x) return true;
    if(x < node->val) return existsHelper(node->left, x);
    return existsHelper(node->right, x);
}

```

B.) exists code →

Time complexity: **O(h)**, which is **O(log n)** for balanced trees, worst-case **O(n)** for skewed trees.

C.) erase code (delete a node in BST code {STRIVER SHEET})

1. **Node not found** → Do nothing.
2. **Node has no children (leaf)** → Just delete it and return **NULL**.
3. **Node has one child** → Replace the node with its child.
4. **Node has two children** → Find the inorder successor (smallest value in the right subtree), replace the current node's value with that, and then delete the successor node recursively.

1. “What happens to memory? Do you need a destructor?”

Expected Answer:

Yes. Since I'm dynamically allocating nodes using **new**, I must free that memory when the object goes out of scope. I should write a destructor that recursively deletes all nodes in the BST.

2. “What about copy constructor and assignment operator?”

Expected Answer:

If I don't define them, the compiler will do a shallow copy of the **root** pointer. That causes double deletion or shared ownership issues. So I need:

- A **copy constructor** (deep copy of all nodes),
- A **copy assignment operator** (free old tree, copy new one),
- Or explicitly disable copying and only allow move semantics.

This is the **Rule of Three** (or Rule of Five if I add move operations).

✓ 3. *“How do you avoid memory leaks?”*

Expected Answer:

Two main risks:

1. Forgetting to `delete` all nodes (fixed with destructor).
2. Reassigning a pointer without deleting old memory first (manage carefully in erase/copy).

I'd also test with tools like Valgrind or print logs to check leaks.

✓ 4. *“What if someone calls `erase(x)` for a value that doesn't exist?”*

Expected Answer:

My recursive erase function already checks for `NULL` at the top, so it safely returns without doing anything.

✓ 5. *“Your BST is unbalanced. What's the worst-case time complexity?”*

Expected Answer:

Worst case: $O(n)$ for insert/erase/find when the tree becomes skewed (like inserting sorted values).

To guarantee $O(\log n)$, we'd need AVL or Red-Black Tree.

✓ 6. *“How would you handle duplicates?”*

Expected Answer:

Since set stores *unique* elements, during insertion, if the value already exists (`exists(x)` is true), I'd skip inserting.

✓ 7. *“What’s the space complexity?”*

Expected Answer:

$O(n)$ for storing n nodes. Each node holds one int and two pointers.

✓ 8. *“Can your data structure iterate elements in sorted order?”*

Expected Answer:

Yes — by simple in-order DFS traversal. That would print or store elements in ascending order.

✓ 9. *“Can your design support iterators like `std::set`?”*

Expected Answer:

Not currently. To support iterators, I'd have to:

- Maintain parent pointers, or
- Do threaded traversal, or
- Store nodes in a sorted list and link them.

But that's additional complexity and not needed for basic implementation.

✓ 10. *“What about `lower_bound` and `upper_bound` efficiency?”*

Expected Answer:

My `lower_bound` runs in $O(\text{height}) \rightarrow O(\log n)$ for balanced, $O(n)$ for skewed. Same logic applies for `upper_bound`.
